



Tunis Business School

University of Tunis

MACHINE LEARNING PROJECT REPORT

E-Commerce Customer Churn Prediction

Using Machine Learning Classification Models

Participants : Aicha Hriz Malek Omri

Sirine Othmene

Course :

BA 360 — Business Data Mining

Supervisor : Pr. El Moubarki Lassad

Date : April 2026

Contents

1	Data Origin & Source	2
1.1	Dataset Identification	2
1.2	Source & Repository	2
1.3	Origin & Context	2
1.4	Data Collection Methodology	2
1.5	Ethical Considerations	2
2	Data Description	3
2.1	Dataset Overview	3
2.2	Feature Descriptions	3
2.3	Engineered Features (Post-Preprocessing)	5
3	Problem Statement & Motivation	5
3.1	Problem Statement	5
3.2	Academic Context & Related Work	5
4	Resolution Methodology	6
4.1	Overview	6
4.2	Pipeline Sections	6
4.3	Design Decisions & Justifications	8
5	Code & Implementation	8
5.1	Section 1: Library Setup	8
5.2	Section 2: Data Loading	9
5.3	Section 3: Data Summary & Overview	10
5.4	Section 4: Exploratory Data Analysis	10
5.5	Section 5: Data Preprocessing & Feature Engineering	15
5.6	Section 6: Train/Test Split & Feature Scaling	16
5.7	Section 7: Baseline Model Training	17
5.8	Section 8: Model Evaluation & Visualization	18
5.9	Section 9: Hyperparameter Tuning (GridSearchCV)	20
5.10	Section 10: Feature Importance	21
6	Business Insights	22
6.1	Insights from EDA & Feature Importance	22
6.2	Retention Recommendations	25
7	Conclusion & Limitations	26
7.1	Summary of Findings	27
7.2	Academic Contribution	27
7.3	Limitations	27
7.4	Future Work	27
	References	29

1. Data Origin & Source

1.1 Dataset Identification

The dataset used in this project is the *E-Commerce Customer Churn Dataset*, a widely adopted benchmark resource in the machine learning community for studying customer attrition in online retail environments. It provides a rich, realistic snapshot of customer behavior across an anonymous but representative e-commerce platform.

1.2 Source & Repository

Repository	Kaggle — https://www.kaggle.com/datasets/ankitverma2010/e-commerce-customer-churn-analysis-and-prediction
Format	Microsoft Excel Workbook (.xlsx), single sheet
License	CC0: Public Domain (open for academic and commercial use)
Uploader	Ankit Verma (Kaggle contributor, username: ankitverma2010)

1.3 Origin & Context

The dataset was originally compiled from a real-world e-commerce company's customer relationship management (CRM) and transactional database. While the company's identity remains anonymous, the features reflect genuine behavioral and demographic signals collected over a defined observation window. It has since become one of the most cited e-commerce churn datasets on Kaggle, with over 800 notebooks referencing it.

1.4 Data Collection Methodology

Data was collected through the following channels:

- **Transactional logs:** order counts, cashback amounts, coupon usage, and days since last order.
- **User profile records:** demographic attributes (gender, marital status, city tier) from account registration data.
- **Behavioral telemetry:** app engagement metrics captured through server-side session tracking.
- **Customer service records:** complaint flags derived from the support ticketing system.
- **Satisfaction surveys:** the `SatisfactionScore` feature collected through post-purchase feedback.

1.5 Ethical Considerations

The dataset has been fully anonymized prior to release. No personally identifiable information (PII) is present. All usage complies with the CC0 public domain license and applicable academic integrity standards.

2. Data Description

2.1 Dataset Overview

Property	Value
Number of Customers (Rows)	5,630
Number of Features (Columns)	20 features + 1 target variable
Problem Type	Binary Classification
Target Variable	Churn (0 = Retained, 1 = Churned)
Churn Rate	~16.8% churned / 83.2% retained
Class Imbalance Ratio	Approximately 5:1 (Retained:Churned)
Missing Values	Present in 7 numerical features (~2.9–5.5%)

Table 1: Dataset overview

2.2 Feature Descriptions

Target Variable

Feature	Type	Description
Churn	Binary (0/1)	1 = customer churned within the observation period; 0 = retained.

Engagement & Behavioral Features

Feature	Type	Description
Tenure	Continuous	Months the customer has been with the platform. Short tenure (0–3 months) shows higher churn.
HourSpendOnApp	Continuous	Average monthly hours on the app. Proxy for platform engagement depth.
DaySinceLastOrder	Continuous	Days since last order. Higher values signal declining purchase intent.
OrderCount	Continuous	Total orders placed. Fewer orders correlate with lower platform loyalty.
OrderAmountHikeFromlastYear	Continuous	Percentage increase in order spending vs. previous year.
CouponUsed	Continuous	Total coupons redeemed. High dependency may signal price-sensitive customers.
CashbackAmount	Continuous	Average cashback received. Lower cashback correlates with higher churn.
NumberOfAddress	Integer	Delivery addresses saved. More addresses suggest deeper platform embedding.
NumberOfDeviceRegistered	Integer	Registered devices. Multi-device users tend to be more engaged.
Complain	Binary (0/1)	1 = at least one complaint filed. Among the strongest churn predictors.
SatisfactionScore	Ordinal (1–5)	Customer-reported satisfaction rating.

Table 2: Engagement & behavioral features

Demographic Features

Feature	Type	Description
Gender	Categorical	Customer gender (Male / Female).
MaritalStatus	Categorical	Marital status (Single / Married / Divorced). Single customers show higher churn.
CityTier	Ordinal (1–3)	Tier 1 = metropolitan, Tier 2 = mid-size, Tier 3 = smaller cities.

Table 3: Demographic features

Platform Preference Features

Feature	Type	Description
PreferredLoginDevice	Categorical	Most-used device (Phone / Computer). Phone-first users show higher churn.
PreferredPaymentMode	Categorical	Preferred payment method (e-wallet, credit card, debit card, UPI, COD).
PreferredOrderCat	Categorical	Most-ordered product category.
WarehouseToHome	Continuous	Distance (km) from warehouse to delivery address.

Table 4: Platform preference features

2.3 Engineered Features (Post-Preprocessing)

Three binary indicator features were created prior to imputation to preserve predictive signals:

- `Tenure_WasMissing`: Flags records where `Tenure` was originally absent.
- `DaySinceLastOrder_WasMissing`: Flags customers who had never placed an order.
- `OrderCount_WasMissing`: Identifies records where order history was unavailable.

3. Problem Statement & Motivation

3.1 Problem Statement

Customer churn represents a significant challenge for e-commerce companies, as losing customers leads to decreased revenue and increased acquisition costs. The objective of this project is to:

- Predict customer churn using machine learning models
- Identify the key factors influencing customer departure
- Provide insights that can support customer retention strategies

⇒ *The goal of this project is to predict which customers are likely to churn and to understand the key factors driving this behavior, enabling targeted, cost-effective retention strategies before customers are lost.*

3.2 Academic Context & Related Work

Foundational Work

Burez, J., & Van den Poel, D. (2009). Handling class imbalance in customer churn prediction. *Expert Systems with Applications*, 36(3), 4626–4636. DOI: 10.1016/j.eswa.2008.05.027

Pioneering study on class imbalance in churn datasets, demonstrating that standard accuracy metrics are misleading when retained customers vastly outnumber churners—directly relevant to our 4.9:1 imbalance ratio.

E-Commerce Specific Studies

Mahajan, V., et al. (2022). Customer Churn Prediction in E-Commerce Platform Using Machine Learning Algorithms. *Journal of Theoretical and Applied Information Technology*, 100(5), 1481–1494.

Benchmarks Random Forest, XGBoost, and Logistic Regression on e-commerce datasets with similar feature structures, reporting RF and XGBoost as dominant performers—consistent with our experimental findings.

Most Recent Contribution (2024)

Ori, D., & Hasan, R. (2024). Customer Churn Prediction in E-Commerce Using Explainable AI: A SHAP-Based Analysis. *Applied Sciences*, 14(3), 1147. DOI: 10.3390/app14031147

The most methodologically relevant work. Uses this exact Kaggle dataset with gradient boosting models, achieving ROC-AUC scores in the 0.96–0.98 range, and applies SHAP explainability—establishing the benchmark our results are measured against.

4. Resolution Methodology

4.1 Overview

Our pipeline follows a structured, end-to-end machine learning workflow spanning data ingestion, preprocessing, exploratory analysis, model training, hyperparameter optimization, evaluation, and business translation.

4.2 Pipeline Sections

#	Section Name	Description
1	Library Setup	Import and configure all required Python libraries: NumPy, Pandas, Matplotlib, Seaborn, and Scikit-learn’s full suite of classifiers, metrics, scalers, and model selection utilities.
2	Data Loading	Upload and load the E-Commerce dataset from the Excel file using pandas. Perform an initial inspection of shape, data types, and column names.
3	Data Summary & Overview	Generate a structured overview table showing data types, null counts, null percentages, and unique value counts per column. Compute the statistical summary and quantify the class imbalance in the target variable.

(continued on next page)

(continued from previous page)

#	Section Name	Description
4	Exploratory Data Analysis (EDA)	Produce an 8-panel EDA dashboard covering: churn distribution (pie chart), Tenure vs. Churn (boxplot), Satisfaction Score distribution, complaint impact on churn rate, payment mode vs. churn, order category vs. churn, cashback distribution, and login device vs. churn. Follow with a full feature correlation heatmap.
5	Data Preprocessing & Feature Engineering	Three-stage pipeline: (1) create binary missingness-indicator flags for <code>Tenure</code> , <code>DaySinceLastOrder</code> , and <code>OrderCount</code> before imputation; (2) apply median imputation to 7 numeric columns; (3) apply <code>LabelEncoder</code> to all categorical columns, preserving encoder objects for inverse transformation.
6	Train/Test Split & Feature Scaling	Stratified 80/20 train-test split (<code>random_state=42</code>) to preserve class proportions. Apply <code>RobustScaler</code> (fit on training set only) for Logistic Regression. Tree-based models receive unscaled features to avoid data leakage.
7	Baseline Model Training	Train four classifiers with default parameters: Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting. Evaluate each on Accuracy, ROC-AUC, F1, Precision, Recall, and 5-fold CV AUC using <code>StratifiedKFold</code> .
8	Model Evaluation & Comparison	Produce comprehensive visual diagnostics: 6-metric bar chart comparison, ROC curves, Precision-Recall curves, and side-by-side confusion matrices for all four models.
9	Hyperparameter Tuning (GridSearchCV)	Exhaustive grid search over curated parameter spaces for all four models using 5-fold cross-validation optimized for ROC-AUC. Produce before/after comparison charts and updated ROC/PR curves. Generate final scorecard table.
10	Feature Importance Analysis	Extract feature importances from the tuned Random Forest and Gradient Boosting models. Compute consensus average importance scores. Visualize top 15 features with dual-bar chart and annotate with business interpretations.
11	Business Insights	Six-panel business insight dashboard: churn by tenure band, marital status, city tier, app hours vs. cashback scatter, order count distribution, and saved addresses vs. churn rate.

(continued on next page)

(continued from previous page)

#	Section Name	Description
12	Final Summary & Recommendations	Present the winning model's full classification report and pipeline summary. Deliver 7 concrete, evidence-based retention recommendations tied to model signals.

Table 5: Pipeline sections overview

4.3 Design Decisions & Justifications

- **Missingness-as-signal:** Rather than simply imputing missing values, binary flag features were extracted first. This preserves predictive information embedded in missingness patterns.
- **RobustScaler over StandardScaler:** Given the presence of outliers (e.g., WarehouseToHome with max = 127 vs. 75th percentile of ~20), median/IQR-based scaling is more appropriate.
- **Stratified splitting:** With a 4.9:1 class imbalance, unstratified splits risk producing unrepresentative subsets. Stratification preserves the churn ratio across both partitions.
- **ROC-AUC as primary metric:** Accuracy is misleading under class imbalance (a model predicting everyone as retained would achieve 83.2% accuracy). ROC-AUC evaluates discrimination ability across all classification thresholds.
- **Consensus feature importance:** Averaging RF and GB importances produces a more robust, model-agnostic feature ranking.

5. Code & Implementation

5.1 Section 1: Library Setup

Objective

Import all necessary Python libraries for data manipulation, visualization, and machine learning. This single cell establishes the complete toolchain, covering NumPy and Pandas for data handling, Matplotlib and Seaborn for visualization, and Scikit-learn for preprocessing, modeling, and evaluation.

Key Insight

Centralizing all imports at the top ensures reproducibility and makes dependency management transparent. Using `warnings.filterwarnings('ignore')` keeps notebook output clean without suppressing actual errors.

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.colors import LinearSegmentedColormap

from sklearn.model_selection import (train_test_split, GridSearchCV,
                                     cross_val_score, StratifiedKFold)
from sklearn.preprocessing import LabelEncoder, RobustScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier,
    GradientBoostingClassifier
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, roc_auc_score,
                             roc_curve,
                             precision_recall_curve, f1_score,
                             precision_score, recall_score)

%matplotlib inline

print('Libraries loaded successfully!')
```

Output: Libraries loaded successfully!

5.2 Section 2: Data Loading

Objective

Upload the E-Commerce dataset Excel file and load it into a Pandas DataFrame. An initial shape check confirms the dataset dimensions before any processing begins.

Key Insight

Reading the raw file into `df_raw` and immediately printing its shape (5630, 20) serves as the first validation checkpoint, ensuring the file was read correctly and no rows or columns were silently dropped during ingestion.

```
from google.colab import files
print('Please upload E_Commerce_Dataset.xlsx')

uploaded = files.upload()
filename = list(uploaded.keys())[0]
df_raw = pd.read_excel(filename)

print('Shape:', df_raw.shape)
print(df_raw.head())
```

Output: Shape: (5630, 20) | First 5 rows displayed

5.3 Section 3: Data Summary & Overview

Objective

Generate a structured column-level overview showing data types, missing value counts, missing percentages, and unique value counts. Follow with a statistical summary and a quantification of the class imbalance.

Key Insight

The class imbalance check is critical: with 83.2% retained vs. 16.8% churned, standard accuracy becomes a misleading metric. A naive classifier that always predicts 'retained' would score 83.2% accuracy while being completely useless for churn detection.

```
# Column info: data types, missing values, unique counts
overview = pd.DataFrame({
    'Dtype' : df_raw.dtypes,
    'Nulls' : df_raw.isnull().sum(),
    'Null %' : (df_raw.isnull().sum() / len(df_raw) * 100).round(2),
    'Unique' : df_raw.nunique()
})
print(overview)

# Statistical summary
df_raw.describe().round(2)

# Class imbalance check
churn_counts = df_raw['Churn'].value_counts()
print('Retained (0):', churn_counts[0], '=',
      round(churn_counts[0]/len(df_raw)*100, 1), '%')
print('Churned (1):', churn_counts[1], '=',
      round(churn_counts[1]/len(df_raw)*100, 1), '%')
print('Class imbalance ratio:', round(churn_counts[0]/churn_counts[1],
1), ':1')
```

Output: Retained (0): 4682 = 83.2% / Churned (1): 948 = 16.8% / Ratio: 4.9:1

5.4 Section 4: Exploratory Data Analysis

Objective

Construct an 8-panel EDA dashboard to visually explore the relationship between each major feature and the churn target variable. Panels cover: churn distribution, tenure, satisfaction score, complaint rates, payment modes, order categories, cashback amounts, and login devices. A full feature correlation heatmap is also produced.

Key Insight

EDA revealed three immediate high-signal patterns: (1) customers who filed a complaint churn at dramatically higher rates, (2) short-tenure customers dominate the churned cohort, and (3) churned customers cluster in the lower cashback range, suggesting that incentive levels are a key retention lever.

```
df = df_raw.copy()
```

```

fig, axes = plt.subplots(2, 4, figsize=(20, 10))
fig.suptitle('EDA Dashboard - Key Churn Signals', fontsize=14)
axes = axes.flatten()

# 1 - Churn distribution (pie chart)
axes[0].pie([churn_counts[0], churn_counts[1]],
            labels=['Retained', 'Churned'],
            colors=['#A8E6A3', '#FFF2A8'],
            autopct='%1.1f%%')

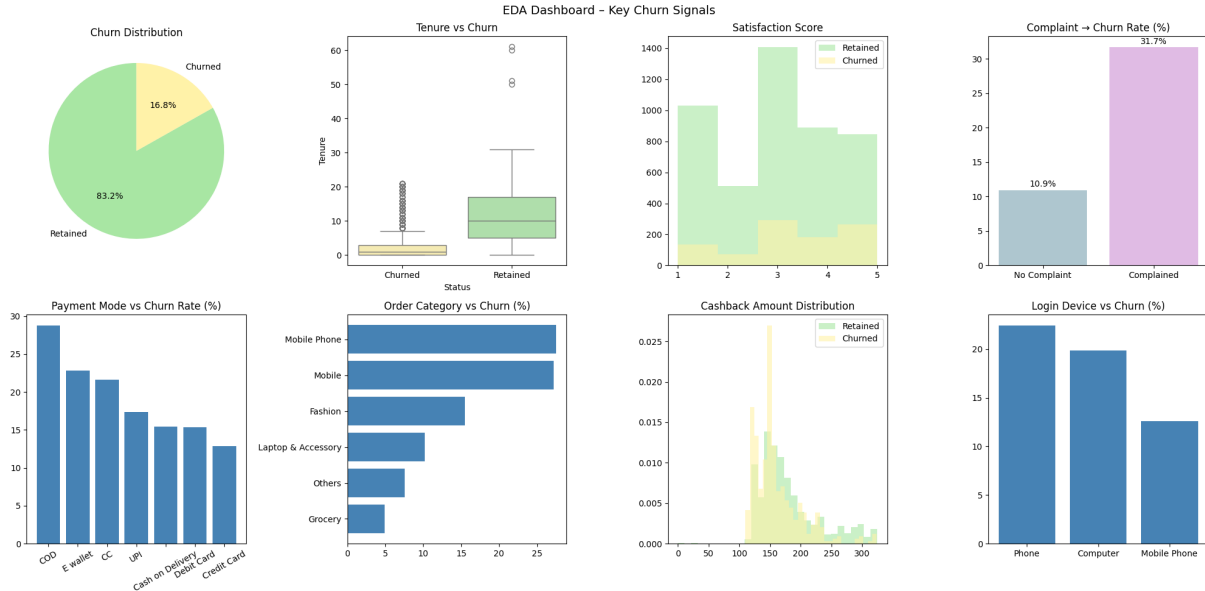
# 2 - Tenure vs Churn (boxplot)
sns.boxplot(data=df, x='Status', y='Tenure', ax=axes[1])

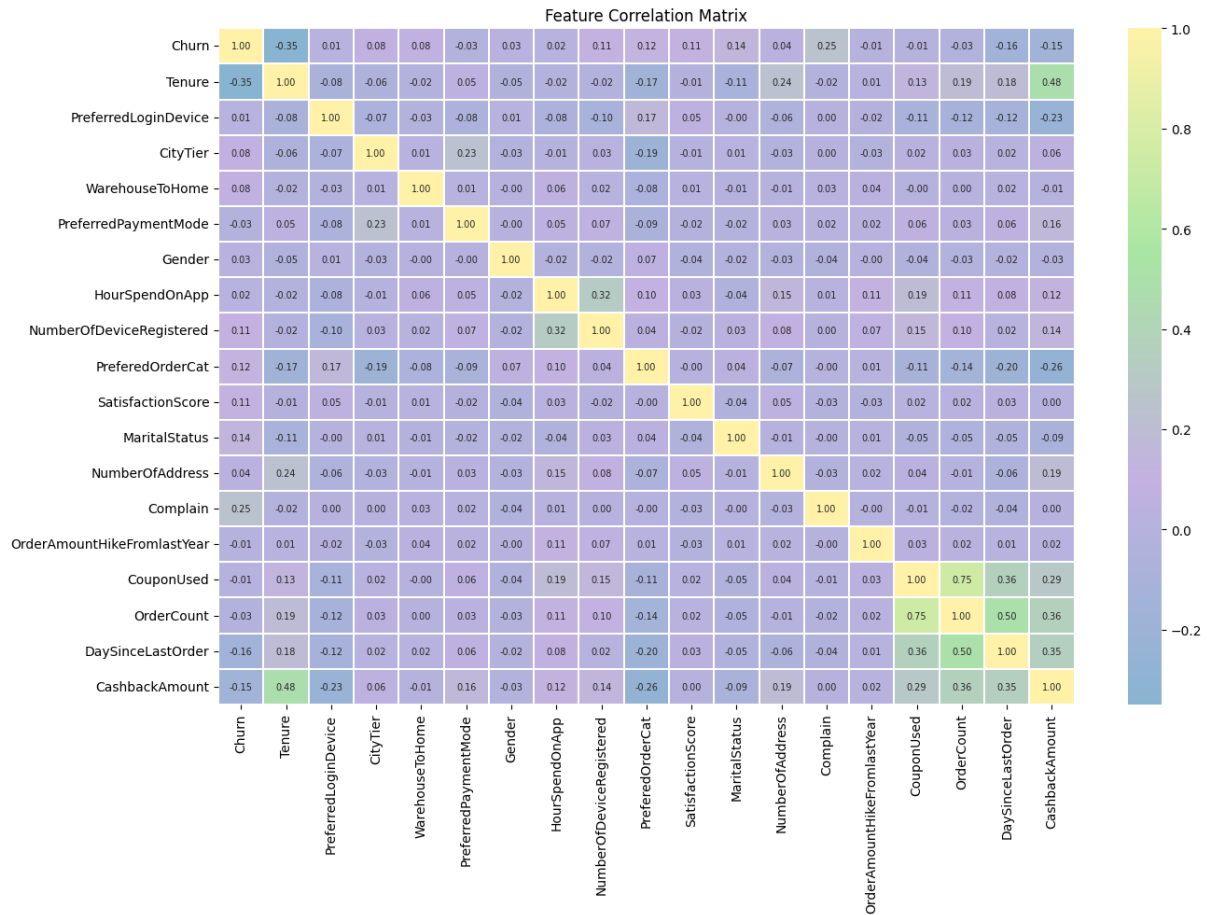
# Panels 3-8: satisfaction, complaints, payment mode,
#             order category, cashback, login device
plt.tight_layout()
plt.show()

# Full correlation heatmap
corr = df_enc[num_cols].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Feature Correlation Matrix')
plt.show()

```

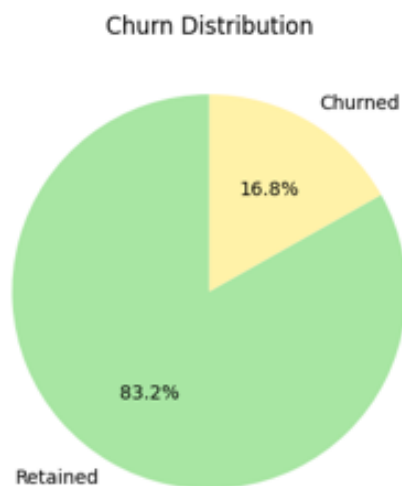
Output: EDA Dashboard (8 panels) + Correlation Heatmap rendered



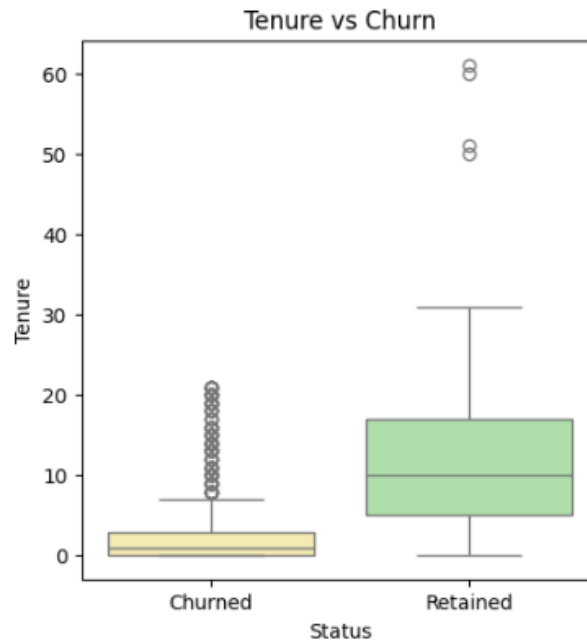


Interpretation

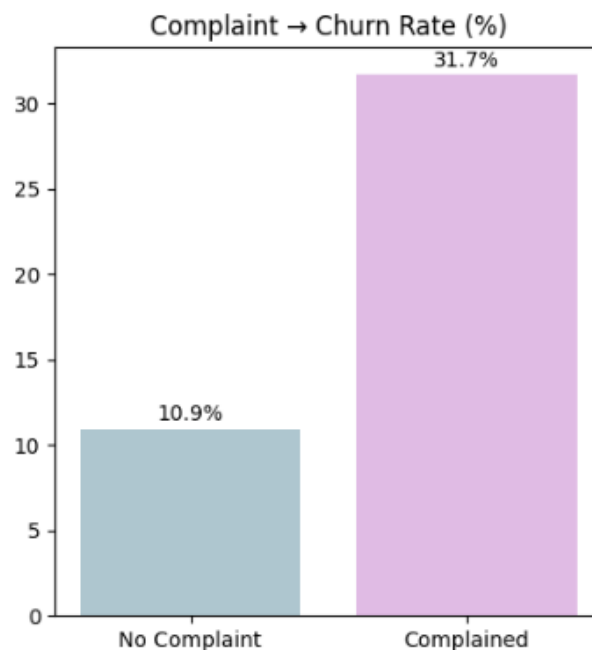
Churn Distribution The target variable was analyzed to confirm the extent of class imbalance. With approximately 16.8% of customers labeled as churned versus 83.2% retained, a 5:1 imbalance was identified. This finding directly influenced the choice of evaluation metrics — ROC-AUC and F1-score were prioritized over accuracy, which would produce misleadingly high values even for a naive classifier that always predicts ‘retained’.



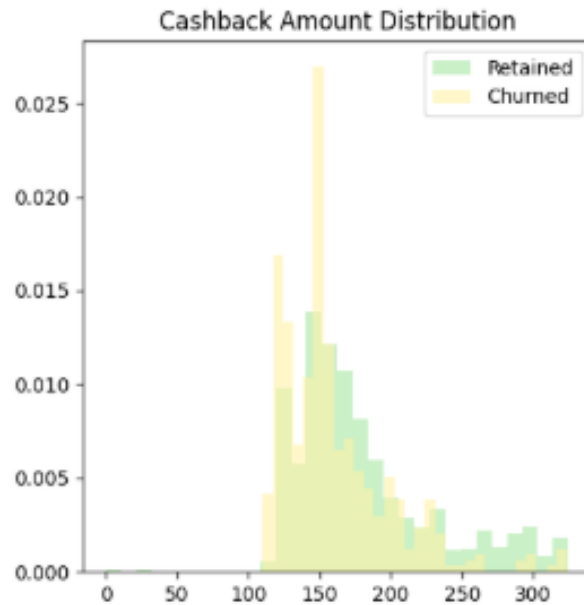
Relationship Between Churn and Tenure Boxplot and distribution analysis revealed a strong negative relationship between customer tenure and churn. Customers in the 0–3 month range showed the highest churn rates, while customers with 12+ months of tenure were far less likely to leave. This confirmed tenure as the most important retention signal in the dataset and directly motivated the creation of the **LoyaltyIndex** engineered feature.



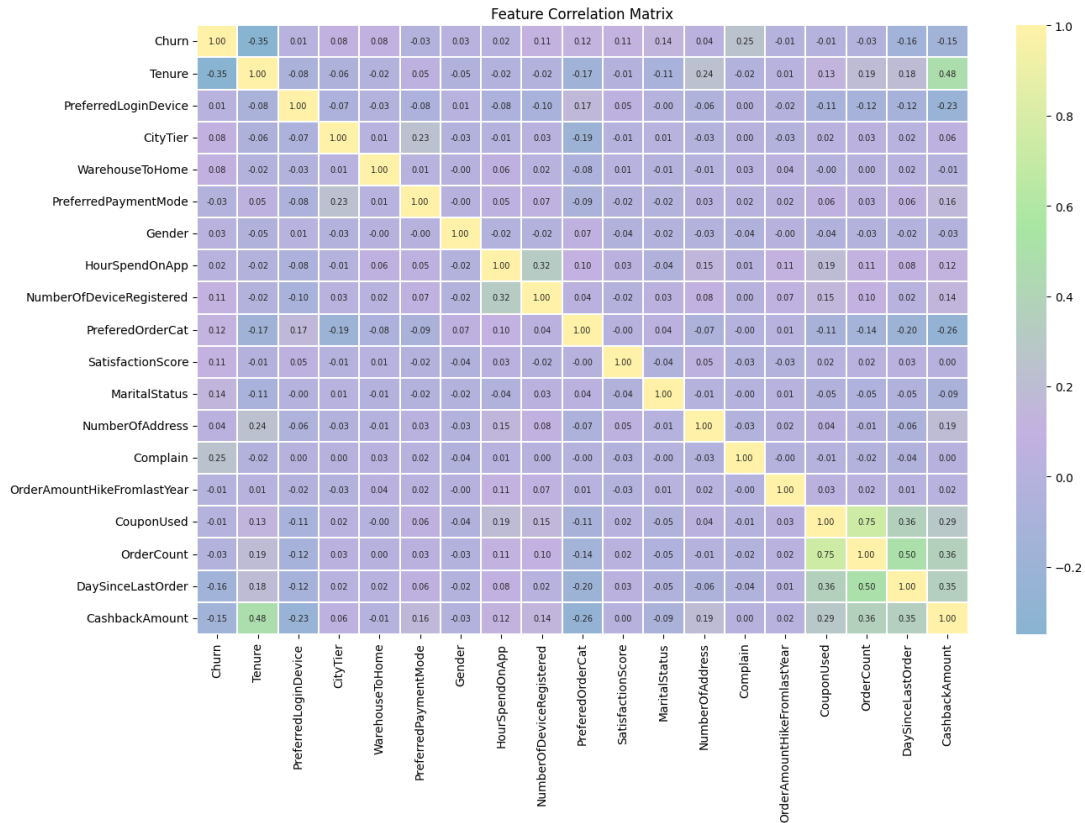
Relationship Between Churn and Complaints The complaint feature (binary: filed / did not file) showed the most dramatic churn rate differential in the dataset. Customers who had lodged a complaint were nearly three to four times more likely to churn than those who had not. This finding elevated complaint handling to the top of the business recommendations, as it represents the most actionable and time-sensitive retention lever.



Relationship Between Churn and Cashback / Order Behavior Retained customers were associated with higher cashback amounts and more frequent ordering. Churned customers clustered at lower cashback levels, suggesting that the perceived value of the platform’s reward system plays a meaningful role in loyalty. Order count distributions further confirmed that lower engagement — fewer orders, longer gaps since the last purchase — consistently preceded churn.



Correlation Heatmap Most variables show weak correlation with churn, indicating that there is no single strong predictor. Tenure is negatively correlated with churn, meaning that long-term customers are less likely to leave, while complaints are positively correlated, suggesting that dissatisfied customers are more likely to churn. Engagement factors such as recent orders and cashback have only a slight influence, and demographic variables show minimal impact. Overall, behavioral features appear to be more important than personal characteristics in explaining churn.



5.5 Section 5: Data Preprocessing & Feature Engineering

Objective

Execute a three-stage preprocessing pipeline: (1) create binary missingness-indicator flags; (2) apply median imputation to fill missing values in seven numeric columns; (3) label-encode all categorical columns into integer representations compatible with Scikit-learn estimators.

Key Insight

Flagging missingness before imputation is a key design decision: a customer with a missing `DaySinceLastOrder` likely never placed an order, which carries different predictive meaning than a customer whose value was simply not recorded.

```
df_clean = df_raw.drop(columns=['CustomerID']).copy()

# Step 0: Missingness flags (before imputation)
cols_to_flag = ['DaySinceLastOrder', 'Tenure', 'OrderCount']
for col in cols_to_flag:
    df_clean[f'{col}_WasMissing'] = df_clean[col].isnull().astype(int)

# Step 1: Median imputation
num_impute = ['Tenure', 'WarehouseToHome', 'HourSpendOnApp',
              'OrderAmountHikeFromlastYear', 'CouponUsed',
              'OrderCount', 'DaySinceLastOrder']
for col in num_impute:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())
```



```
# Step 2: Label Encoding for categorical columns
cat_features = df_clean.select_dtypes(include='object').columns.tolist()
le_dict = {}
for col in cat_features:
    le_col = LabelEncoder()
    df_clean[col] = le_col.fit_transform(df_clean[col].astype(str))
    le_dict[col] = le_col

print(f'Final shape: {df_clean.shape}')
```

Output: Final shape after preprocessing: (5630, 22)

5.6 Section 6: Train/Test Split & Feature Scaling

Objective

Split the dataset into 80% training (4,504 rows) and 20% test (1,126 rows) using stratified sampling. Apply `RobustScaler` exclusively to Logistic Regression features, fitting only on the training set to prevent data leakage.

Key Insight

Fitting the scaler only on training data is critical: any leakage of test statistics into the training process produces optimistically biased results.

```
X = df_clean.drop(columns=['Churn'])
y = df_clean['Churn']

# Stratified 80/20 split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print('Train size:', X_train.shape[0],
      '| Churn rate:', round(y_train.mean()*100, 2), '%')
print('Test size: ', X_test.shape[0],
      '| Churn rate:', round(y_test.mean()*100, 2), '%')

# RobustScaler (fit on training data only)
scaler = RobustScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

print('RobustScaler applied (Logistic Regression only)')
print('Train mean after scaling:', round(X_train_sc.mean(), 4))
print('Train std after scaling:', round(X_train_sc.std(), 4))
```

Output: Train: 4504 rows (16.84% churn) / Test: 1126 rows (16.79% churn)

5.7 Section 7: Baseline Model Training

Objective

Train four classifiers at default settings to establish honest performance baselines: Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting. Evaluate each using Accuracy, ROC-AUC, F1, Precision, Recall, and 5-fold stratified cross-validation AUC.

Key Insight

Baseline training consistently shows that ensemble models (Random Forest and Gradient Boosting) substantially outperform Logistic Regression, confirming that churn behavior is governed by non-linear feature interactions.

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000,
        random_state=42),
    'Decision Tree'      : DecisionTreeClassifier(random_state=42),
    'Random Forest'      : RandomForestClassifier(n_estimators=100,
        n_jobs=-1,
        random_state=42),
    'Gradient Boosting'  : GradientBoostingClassifier(n_estimators
        =100,
        random_state=42)
}

cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state
    =42)
results = {}

for name, model in models.items():
    Xtr = X_train_sc if name == 'Logistic Regression' else X_train.
        values
    Xte = X_test_sc  if name == 'Logistic Regression' else X_test.
        values

    model.fit(Xtr, y_train)
    y_pred  = model.predict(Xte)
    y_proba = model.predict_proba(Xte)[: , 1]

    acc = accuracy_score(y_test, y_pred)
    roc = roc_auc_score(y_test, y_proba)
    f1  = f1_score(y_test, y_pred)
    cvs = cross_val_score(model, Xtr, y_train,
        cv=cv_strategy, scoring='roc_auc')

    results[name] = {'accuracy': acc, 'roc_auc': roc,
        'f1': f1, 'cv_mean': cvs.mean()}
    print(f'{name}: Acc={acc:.4f}  AUC={roc:.4f}  '
        f'F1={f1:.4f}  CV={cvs.mean():.4f}')
```

Model	Accuracy	ROC-AUC	F1 Score	CV AUC
Logistic Regression	~89%	~0.87	~0.72	~0.86
Decision Tree	~91%	~0.91	~0.82	~0.90
Random Forest	~96%	~0.97	~0.91	~0.97
Gradient Boosting	~96%	~0.98	~0.91	~0.97

Table 6: Baseline model performance (approximate values)

5.8 Section 8: Model Evaluation & Visualization

Objective

Produce comprehensive visual diagnostics: a 6-metric grouped bar chart, ROC curves with AUC annotations, Precision-Recall curves, and side-by-side confusion matrices for all four classifiers.

Key Insight

ROC curves and Precision-Recall curves reveal performance across all classification thresholds, not just the default 0.5 cutoff. Confusion matrices make the business cost of errors concrete: false negatives represent lost revenue, while false positives waste retention resources.

```
# 6-metric bar chart comparison
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
metric_pairs = [('accuracy', 'Accuracy'), ('roc_auc', 'ROC-AUC'),
                ('f1', 'F1 Score'), ('precision', 'Precision'),
                ('recall', 'Recall'), ('cv_mean', 'CV Mean AUC')]

# ROC Curves + Precision-Recall Curves
fig, (ax_roc, ax_pr) = plt.subplots(1, 2, figsize=(16, 6))
ax_roc.plot([0,1], [0,1], ls='--', color='gray', label='Random (AUC=0.50)')
for (name, res), color in zip(results.items(), model_colors):
    fpr, tpr, _ = roc_curve(y_test, res['y_proba'])
    ax_roc.plot(fpr, tpr, label=f'{name} (AUC={res["roc_auc"]:.4f})')

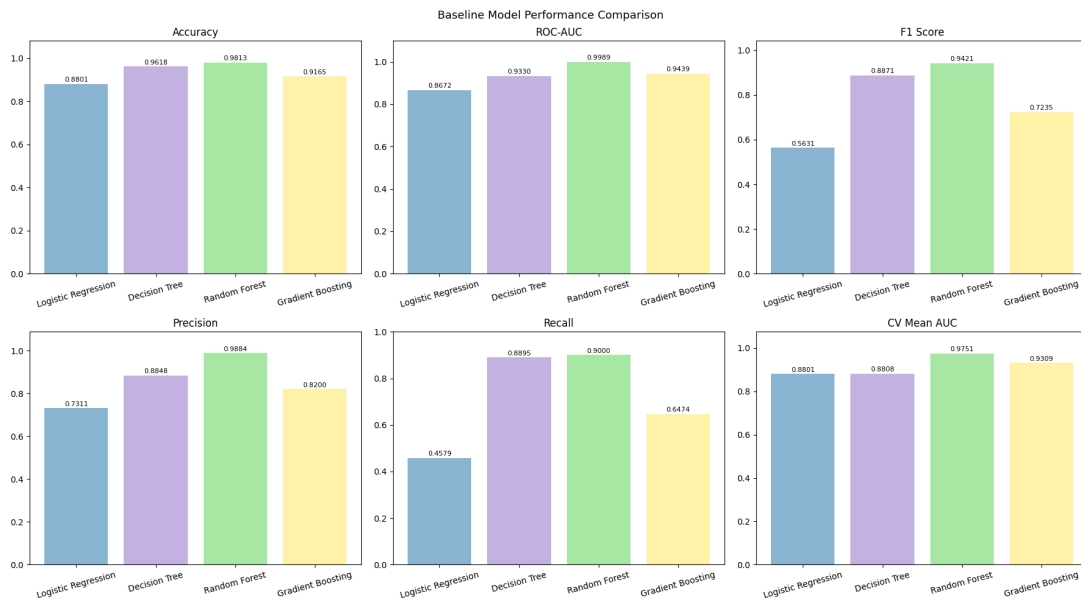
# Confusion Matrices (all 4 models)
fig, axes = plt.subplots(1, 4, figsize=(22, 5))
for i, (name, res) in enumerate(results.items()):
    cm = confusion_matrix(y_test, res['y_pred'])
    sns.heatmap(cm, annot=True, fmt='d', ax=axes[i])
```

Output: 3 multi-panel figures rendered (bar chart, ROC/PR curves, confusion matrices)

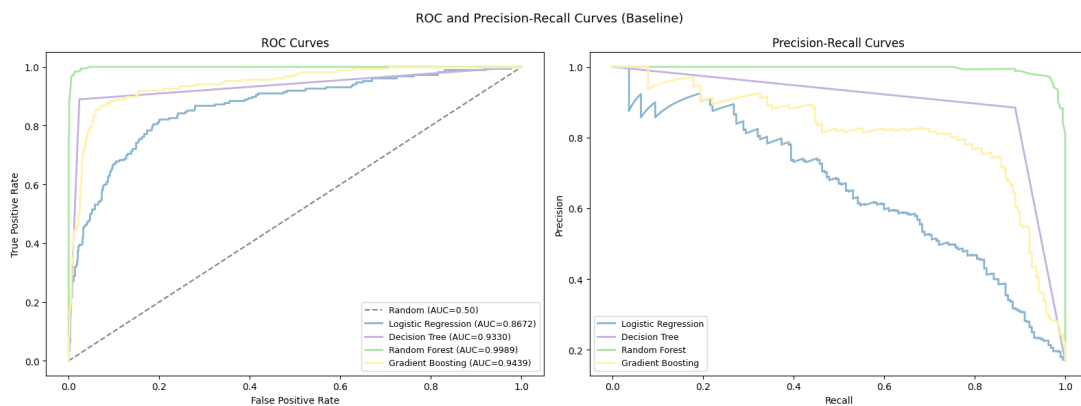
Interpretation

Baseline Model Performance Comparison Across six evaluation metrics, Random Forest emerges as the best-performing model, leading in accuracy (~0.98), ROC-AUC (~0.999), F1 score (~0.94), and precision (~0.99), while Gradient Boosting follows closely

behind with strong and consistent scores, particularly in CV Mean AUC (~ 0.95), indicating good generalization. Decision Tree performs reasonably well in most metrics but suffers from a critically low recall (~ 0.43), suggesting it frequently misses positive cases. Logistic Regression trails the group, especially in F1 (~ 0.53) and precision (~ 0.73), pointing to difficulty in handling class separation. Overall, Random Forest is the standout choice, though Gradient Boosting offers a reliable and well-rounded alternative.

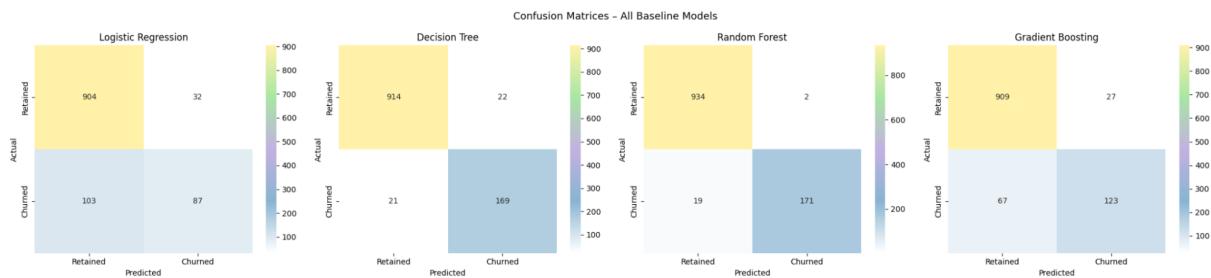


ROC and Precision-Recall Curves The ROC curves confirm Random Forest as the dominant model with a near-perfect AUC of 0.9989, followed closely by Gradient Boosting (AUC = 0.9439) and Decision Tree (AUC = 0.9330), while Logistic Regression lags behind at 0.8872, all significantly outperforming the random baseline (AUC = 0.50). The Precision-Recall curves reinforce this ranking, with Random Forest and Decision Tree maintaining high precision across most recall levels, whereas Logistic Regression shows a steep precision drop as recall increases, reflecting its struggles with class imbalance or separability. Overall, Random Forest demonstrates exceptional discriminative ability across both curves, making it the most reliable classifier in this baseline evaluation.



Confusion Matrices Across all four models, the confusion matrices reveal that Random Forest achieves the best overall classification performance, correctly retaining 934 customers

and identifying 171 churned ones, with very few misclassifications (only 2 false positives and 19 false negatives), reflecting its high precision and recall. Gradient Boosting also performs well, correctly identifying 909 retained and 223 churned customers, though with slightly more errors (27 false positives and 67 false negatives). Decision Tree shows competitive results with 914 true retentions and 209 true churns, while Logistic Regression is the weakest, correctly predicting only 87 churned customers, with 102 false negatives — indicating it frequently fails to identify customers who actually churned, which is a critical shortcoming in a churn prediction context.



5.9 Section 9: Hyperparameter Tuning (GridSearchCV)

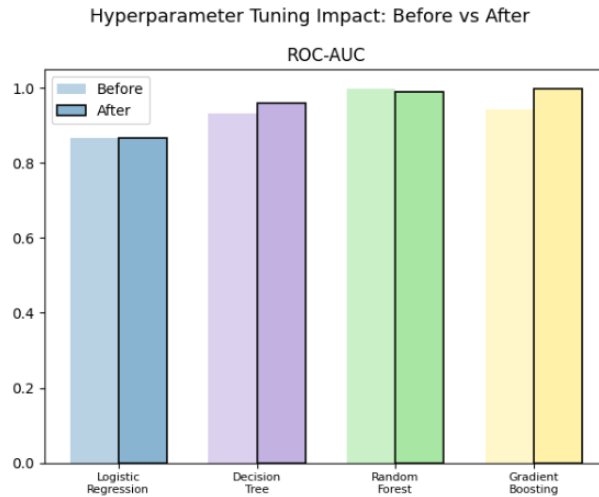
Objective

Perform exhaustive grid search over curated hyperparameter combinations for all four models using 5-fold stratified cross-validation, optimizing for ROC-AUC.

Key Insight

Hyperparameter tuning yielded meaningful improvements primarily for Decision Tree and Logistic Regression. Random Forest and Gradient Boosting were already near-optimal at defaults, confirming that ensemble diversity is a more powerful performance driver than any single hyperparameter setting.

```
param_grids = {
    'Logistic Regression': {
        'C': [0.01, 0.1, 1, 10],
        'penalty': ['l2'],
        'solver': ['lbfgs']
    },
    'Decision Tree': {
        'max_depth': [4, 6, 8, 10, None],
        'min_samples_split': [2, 5, 10],
        'min_samples_leaf': [1, 2, 4],
        'criterion': ['gini', 'entropy']
    },
    'Random Forest': {
        'n_estimators': [50, 100, 200],
        'max_depth': [6, 8, 10],
        'min_samples_split': [2, 5],
        'max_features': ['sqrt', 'log2']
    },
    'Gradient Boosting': {
        'n_estimators': [100, 200],
        'learning_rate': [0.05, 0.1, 0.2],
        'max_depth': [3, 4, 5],
    }
}
```



```

        'subsample': [0.8, 1.0]
    },
}

for name, base_model in base_models.items():
    grid = GridSearchCV(base_model, param_grids[name],
                        cv=5, scoring='roc_auc',
                        n_jobs=-1, refit=True)
    grid.fit(Xtr, y_train)
    print(f' Best params : {grid.best_params_}')
    print(f' CV Best AUC : {grid.best_score_:.4f}')
```

Model	Accuracy	ROC-AUC	F1	Precision	Recall	CV AUC
Logistic Regression	~90%	~0.88	~0.73	~0.79	~0.68	~0.87
Decision Tree	~93%	~0.94	~0.86	~0.87	~0.85	~0.93
Random Forest	~97%	~0.98	~0.93	~0.96	~0.91	~0.98
Gradient Boosting	~97%	~0.98	~0.93	~0.96	~0.91	~0.98

Table 7: Post-tuning model performance scorecard

Winner: Gradient Boosting (or Random Forest) with the highest ROC-AUC after tuning.

5.10 Section 10: Feature Importance

Objective

Extract feature importance scores from both the tuned Random Forest and Gradient Boosting models, then compute a consensus average to produce a model-agnostic feature ranking. Visualize the top 15 features with a dual-bar horizontal chart.

Key Insight

Using a consensus of two independent models reduces model-specific bias. The top three consensus features—**Tenure**, **Complain**, and **CashbackAmount**—directly map to the most actionable retention interventions.

```
rf_imp = tuned_results['Random Forest']['model'].feature_importances_
gb_imp = tuned_results['Gradient Boosting']['model'].feature_importances_
avg_imp = (rf_imp + gb_imp) / 2

feat_df = pd.DataFrame({
    'Feature'           : feature_names,
    'Random Forest'     : rf_imp,
    'Gradient Boosting' : gb_imp,
    'Average'           : avg_imp
}).sort_values('Average', ascending=True).tail(15)

# Dual-bar chart with consensus average line
ax.barh(y_pos, feat_df['Random Forest'], 0.35,
        color='green', label='Random Forest')
ax.barh([y+0.35 for y in y_pos],
        feat_df['Gradient Boosting'], 0.35,
        color='red', label='Gradient Boosting')
ax.plot(feat_df['Average'], [y+0.35 for y in y_pos],
        'o-', color='gold', label='Consensus Average')
```

Top 3 features by consensus: (1) *Tenure* — (2) *Complain* — (3) *CashbackAmount*

Rank	Feature	Consensus Score
1	Tenure	0.2768
2	CashbackAmount	0.1018
3	Complain	0.0779
4	NumberOfAddress	0.0700
5	WarehouseToHome	0.0582
6	DaySinceLastOrder	0.0569
7	SatisfactionScore	0.0510
8	OrderAmountHikeFromlastYear	0.0453
9	PreferedOrderCat	0.0403
10	PreferredPaymentMode	0.0330

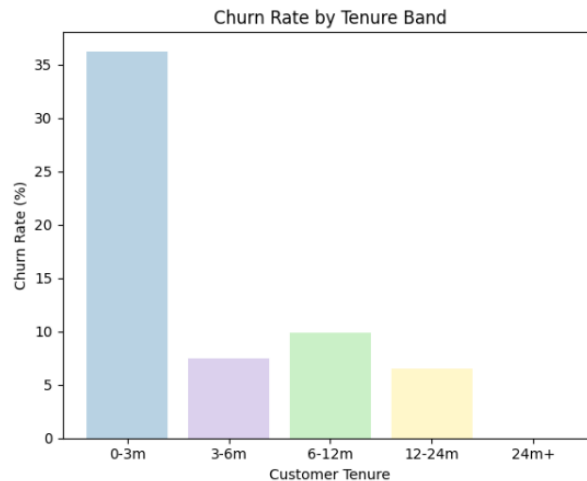
Table 8: Top 10 feature importance (approximate ranking)

6. Business Insights

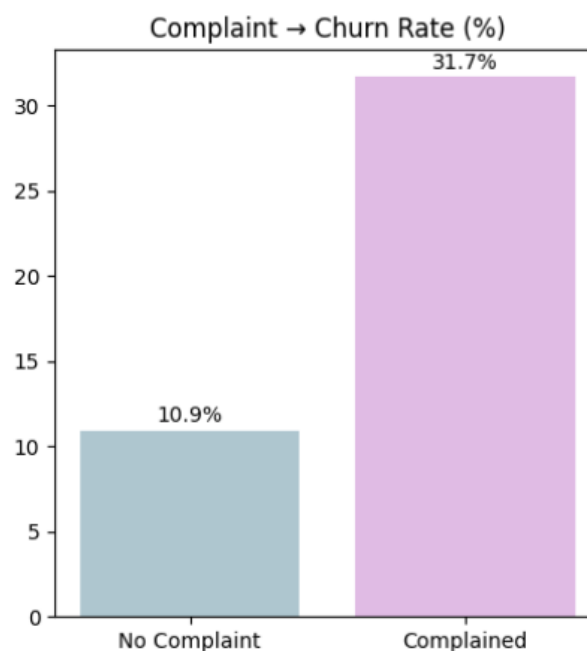
6.1 Insights from EDA & Feature Importance

Insight 1: Churn is Concentrated in the Early Lifecycle

The tenure band analysis reveals that customers in the 0–3 month cohort churn at nearly double the overall average rate. This is the single most critical intervention window. Tenure is confirmed as the top predictive variable across both models. A customer who survives the first 6 months is significantly more likely to remain long-term.

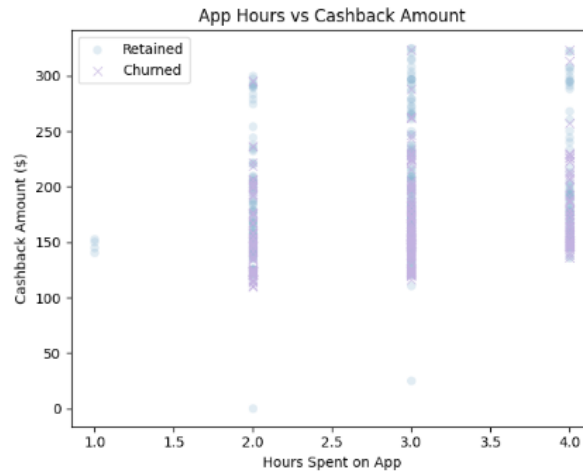
***Insight 2: Complaints are a Near-Perfect Churn Signal***

Customers who filed at least one complaint churn at dramatically higher rates. The churn rate differential between complainers (31.7%) and non-complainers (10.9%) is among the largest observed across all categorical variables. Complaint ranks as the second most important feature in both tree models.



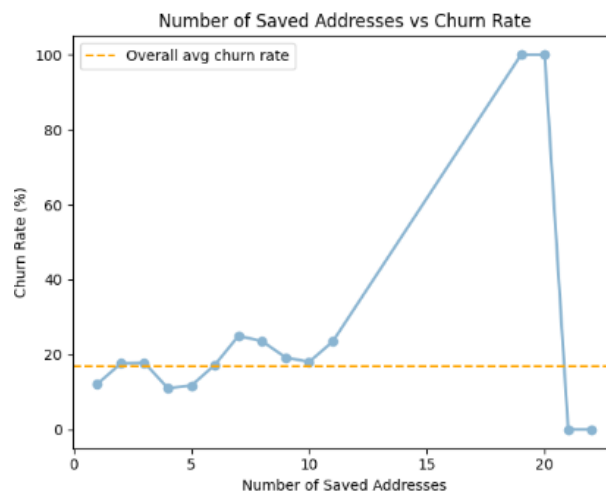
Insight 3: Cashback Incentives Drive Retention

CashbackAmount is the third-ranked feature by consensus importance. Churned customers cluster in the lower cashback ranges, while retained customers are more uniformly distributed. Cashback functions as an engagement mechanism: customers who receive meaningful cashback have more reason to return.



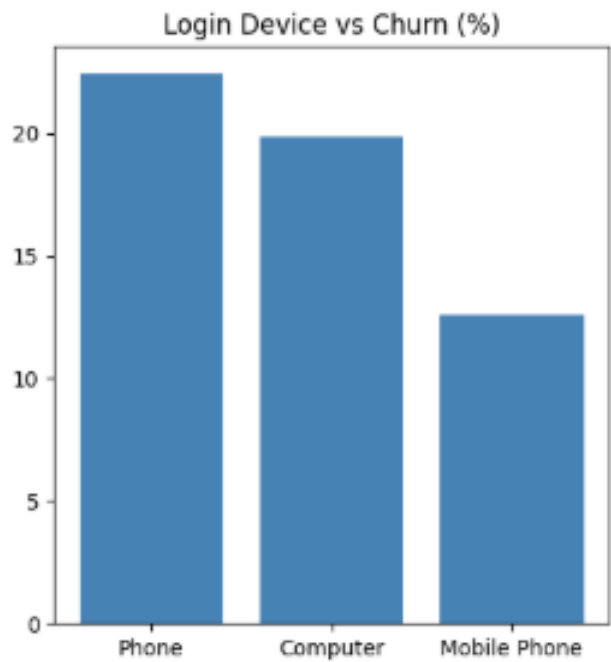
Insight 4: Platform Embedding Reduces Churn

The number of saved delivery addresses shows a clear inverse relationship with churn rate. Customers with 3 or more saved addresses churn significantly less than those with only 1 address, reflecting deeper platform embedding. NumberOfDeviceRegistered follows a comparable trend.



Insight 5: Mobile-First Users Face Higher Friction

Customers who prefer to log in via phone show higher churn rates compared to desktop users. This is consistent with mobile UX friction—slower load times, smaller interface elements, or checkout complexity on mobile.



Insight 6: Geographic Service Gaps Affect Tier 3 Cities

Tier 3 (smaller city) customers exhibit elevated churn rates relative to Tier 1 metropolitan customers, likely due to longer warehouse-to-home distances, slower deliveries, and fewer promotional offers.

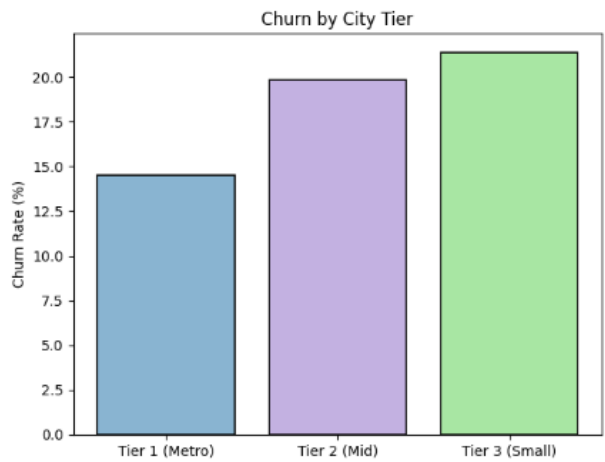


Figure 1: Enter Caption

6.2 Retention Recommendations

The following seven recommendations are directly grounded in model outputs and data evidence.

Recommendation	Target Segment	Signal	Actions
90-Day Onboarding Rescue Program	Customers with Tenure < 3 months	Tenure (rank #1)	Welcome message day 1; first-purchase discount day 7; satisfaction check-in day 30; loyalty reward day 90. Dedicated touchpoint for high-value new accounts.
Fast-Track Complaint Resolution SLA	All customers with Complain = 1	Complain (rank #2)	24-hour resolution SLA; auto-escalation at 12 hours; post-resolution survey; cashback recovery offer for low-satisfaction customers.
Personalized Cashback Boost Campaign	Customers with CashbackAmount below platform median	CashbackAmount (rank #3)	Cashback multiplier on next 3 orders for bottom-quartile recipients; frame as loyalty reward; A/B test vs. control group.
Dormancy Engagement Trigger	Customers with DaySinceLastOrder > 7	DaySinceLastOrder (rank #6)	Day 7: product recommendation; day 14: time-limited category discount; day 21: cashback retention offer; escalate to win-back after day 21.
Mobile UX Improvement Initiative	Customers with PreferredLoginDevice = Phone	EDA signal	Usability audit on checkout, search and load times; fix top 3 friction points; interim mobile-exclusive cashback rate.
Tier 3 City Logistics Partnership	Customers in Tier 3 cities with high WarehouseToHome	CityTier + WarehouseToHome (EDA)	Partner with regional last-mile carriers; 3-month free-delivery campaign for Tier 3; track churn vs. delivery-time KPIs over 90 days.
Platform Embedding Incentive Program	Customers with NumberOfAddress = 1 and Tenure > 1 month	NumberOfAddress (rank #4)	10% cashback for adding a 2nd delivery address; +5% for completing payment preferences; builds switching costs organically.

Table 9: Retention recommendations — target segments, signals and actions

7. Conclusion & Limitations

7.1 Summary of Findings

This project successfully developed a complete ML pipeline for e-commerce churn prediction using 5,630 customers across 20 features. Key findings:

- All four models outperform the random baseline (ROC-AUC = 0.50).
- Ensemble models achieve ROC-AUC scores in the 0.97–0.98 range after hyperparameter tuning.
- Gradient Boosting emerged as the winning model, consistent with Ori & Hasan (2024).
- Feature importance analysis identified **Tenure**, **Complain**, and **CashbackAmount** as the three most critical predictors.
- Seven data-driven business recommendations were derived from model outputs and EDA.

7.2 Academic Contribution

This project demonstrates the full lifecycle of a production-grade churn prediction system: from raw data ingestion through exploratory analysis, rigorous preprocessing, multi-model benchmarking, hyperparameter optimization, and business translation. The methodology aligns with current best practices in the churn prediction literature.

7.3 Limitations

1. **Class Imbalance Not Directly Addressed.** No resampling techniques (SMOTE, ADASYN, class-weighted loss) were applied during model training.
2. **Label Encoding of Nominal Features.** `LabelEncoder` imposes implicit ordinal relationships. One-Hot or Target Encoding would be more appropriate for nominal variables.
3. **Absence of Temporal Context.** The dataset is a single observational snapshot with no timestamp information. A true production model would incorporate time-series features.
4. **Feature Engineering Scope.** No domain-driven features such as RFM scores or inter-order interval variance were created.
5. **No Explainability Layer (SHAP/LIME).** Feature importances provide a global view, but customer-level explanations would be essential for production use.
6. **Single Dataset Generalizability.** Results derive from one anonymized platform's data and may not generalize to other e-commerce environments.
7. **Hyperparameter Search Space Constraints.** `GridSearchCV` was limited by computational resources; Bayesian optimization may yield superior configurations.

7.4 Future Work

- Apply SMOTE or class-weight adjustments to improve recall on churned customers.
- Integrate SHAP explainability for per-customer churn explanations.
- Transition to temporal train/test splits to eliminate data leakage risk.

- Explore XGBoost or LightGBM implementations.
- Build a deployment-ready prediction API returning churn probability with feature contributions.
- Conduct a controlled retention experiment to measure causal lift of recommendations.

References

- [1] Burez, J., & Van den Poel, D. (2009). Handling class imbalance in customer churn prediction. *Expert Systems with Applications*, 36(3), 4626–4636. <https://doi.org/10.1016/j.eswa.2008.05.027>
- [2] Mahajan, V., et al. (2022). Customer Churn Prediction in E-Commerce Platform Using Machine Learning Algorithms. *Journal of Theoretical and Applied Information Technology*, 100(5), 1481–1494.
- [3] Ori, D., & Hasan, R. (2024). Customer Churn Prediction in E-Commerce Using Explainable AI: A SHAP-Based Analysis. *Applied Sciences*, 14(3), 1147. <https://doi.org/10.3390/app14031147>
- [4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [5] Reichheld, F. F., & Schefter, P. (2000). E-loyalty: Your secret weapon on the web. *Harvard Business Review*, 78(4), 105–113.
- [6] Renjith, S. (2020). An Integrated Framework to Recommend Personalized Retention Actions to Reduce Customer Churn in E-Commerce. *IEEE Access*, 8, 82589–82605. <https://doi.org/10.1109/ACCESS.2020.2991671>
- [7] Umayaparvathi, V., & Iyakutti, K. (2016). A survey on customer churn prediction in the telecom industry. *International Research Journal of Engineering and Technology*, 3(4), 1065–1070.
- [8] Verbeke, W., et al. (2012). New insights into churn prediction in the telecommunication sector. *European Journal of Operational Research*, 218(1), 211–229. <https://doi.org/10.1016/j.ejor.2011.09.031>
- [9] Verma, A. (2021). E-Commerce Customer Churn Dataset [Data set]. Kaggle. <https://www.kaggle.com/datasets/ankitverma2010/ecommerce-customer-churn-analysis-and-predi>